Cap 02 Invocação de uma syscall (em C) (Slide 18/22) C

```c
#include <unistd.h>
int main (int argc, char *argv[])
{
    write (1, "Hello World!\n", 13) ;    /* write string to stdout */
    _exit (0) ;                          /* exit with no error */
}
```

Cap 02 Invocação de uma syscall (assembly) (Slide 19/22) ASSEMBLY

```assembly
section .data
    msg db 'Hello World!', 0xA       ; output string (0xA = "\n")
    len equ 13                       ; string size
section .text
global _start
_start:
    mov rax, 1                       ; syscall opcode (1: write)
    mov rdi, 1                       ; file descriptor (1: stdout)
    mov rsi, msg                     ; data to write
    mov rdx, len                     ; number of bytes to write
    syscall                          ; make syscall
    mov rax, 60                      ; syscall opcode (60: _exit)
    mov rdi, 0                       ; exit status (0: no error)
    syscall                          ; make syscall
```

Cap 05 TCB no Linux 1.0 (include/linux/sched.h) (Slide 6/29) C

```c
struct task_struct {
    volatile long state;                /* -1 unrunnable, 0 runnable, >0
    long counter;
    long priority;
    unsigned long signal;
    unsigned long blocked;              /* bitmap of masked signals */
    unsigned long flags;                /* per process flags, defined be
    int errno;
    int debugreg[8];                    /* Hardware debugging registers
    ...
    struct task_struct *next_task, *prev_task;
    struct sigaction sigaction[32];
    int exit_code, exit_signal;
    ...
    unsigned long start_code, end_code, end_data, start_brk, brk,
    start_stack, start_mmap;
    unsigned long arg_start, arg_end, env_start, env_end;
    ...
    struct wait_queue *wait_chldexit;   /* for wait4() */
    unsigned short uid, euid, suid;
    unsigned short gid, egid, sgid;
    ...
    int tty;                            /* -1 if no tty, so it must be s
    unsigned short umask;
    struct inode * pwd;
    ...
    struct vm_area_struct *stk_vma;
};
```

Cap 05 Criação de processos (Slide 12/29) C

```
Processos são criados através de chamadas de sistema.
Em Windows:
CreateProcess: cria um novo processo, informando
o arquivo contendo o código a executar.
Em UNIX:
fork(): duplica o processo atual.
execve(file): substitui o código executável do processo
por outro código.
```

Cap 05 Criação de processos em UNIX (Slide 14/29) C

```c
int main (int argc, char *argv[], char *envp[])
{
    int ret ;
    ret = fork () ;                     // replicação do processo
    if ( ret < 0 ) {
          // fork não funcionou
        perror ("Erro: ") ;
        exit (-1) ;
    }
else
    if ( ret > 0 )                      // sou o processo pai
    wait (0) ;                          // esperar filho concluir
else {
          // sou o processo filho
        execve ("/bin/date", argv, envp) ;   // carrega binário
        perror ("Erro: ") ;                  // execve não funcionou
    }
    printf ("Tchau !\n") ;
    exit(0) ;
}
```

Cap 05 Programando com threads em C (Slide 26/29) C

```c
#include <pthread.h>
...
#define NUM_THREADS 5
void *print_hello (void *threadid)   // cada thread executa esta função
{
    printf ("%ld: Hello World!\n", (long) threadid);
    sleep (5) ;
    printf ("%ld: Bye bye World!\n", (long) threadid);
    pthread_exit (NULL);                 // encerra esta thread
}
int main (int argc, char *argv[])    // thread "main"
{
    pthread_t thread[NUM_THREADS];
    long status, i;
    for(i = 0; i < NUM_THREADS; i++)     // cria as demais threads
    {
        printf ("Creating thread %ld\n", i);
        status = pthread_create (&thread[i], NULL, print_hello, (void *)
        if (status) {
              // ocorreu um erro
            perror ("pthread_create");
            exit (-1);
        }
    }
    pthread_exit (NULL);                 // encerra a thread "main"
}
```

Cap 05 Programando com threads em Java (Slide 27/29) JAVA

```java
public class MyThread extends Thread {
    int threadID;
    private static final int NUM_THREADS = 5 ;
    MyThread (int ID) {
        threadID = ID;
    }
    public void run () {
          // corpo de cada thread
        System.out.println (threadID + ": Hello World!") ;
        try {
            Thread.sleep(5000);
        }
    catch (InterruptedException e) {
            e.printStackTrace();
        }
        System.out.println (threadID + ": Bye bye World!") ;
    }
    public static void main (String args[]) {
        MyThread[] t = new MyThread[NUM_THREADS] ;
        for (int i = 0; i < NUM_THREADS; i++) {
              // cria as threads
            t[i] = new MyThread (i);
        }
        for (int i = 0; i < NUM_THREADS; i++) {
              // inicia a execução das threads
            t[i].start () ;
        }
    }
}
```

Cap 09 Filas POSIX - receptor (Slide 7/12) C

```c
#define QUEUE "/my_queue"
mqd_t queue ;                           // descritor da fila de mensagens
struct mq_attr attr ;                   // atributos da fila de mensagens
int
msg ;                                   // as mensagens são números inteiros
  // abre ou cria a fila com permissoes 0666
if ((queue = mq_open (QUEUE, O_RDWR|O_CREAT, 0666, &attr)) < 0)
{
    perror ("mq_open") ;
    exit (1) ;
}                                       // recebe cada mensagem e imprime se
for (;;)
{
    if ((mq_receive (queue, (void*) &msg, sizeof(msg), 0)) < 0)
    perror ("mq_receive:") ;
else
    printf ("Received msg value %d\n", msg) ;
}
```

Cap 09 Filas POSIX - emissor (Slide 8/12) C

```c
#define QUEUE "/my_queue"           // abre a fila de mensagens, se exis
if ((queue = mq_open (QUEUE, O_RDWR)) < 0)
{
    perror ("mq_open") ;
    exit (1) ;
}
for (;;) {
    msg = random() % 100 ;              // valor entre 0 e 99
      // envia a mensagem
    if (mq_send (queue, (void*) &msg, sizeof(msg), 0) < 0)
    {
        perror ("mq_send") ;
        exit (1) ;
    }
    printf ("Sent message with value %d\n", msg) ;
    sleep (1) ;
}
```